

API Security Risk Analysis

Read-only assessment of public demonstration APIs

Field	Detail
Prepared by	Idrisa Haruni Kigaile
Report title	API Security Risk Analysis Report
Assessment type	Read-only, non-intrusive API security risk analysis
Targets in scope	jsonplaceholder.typicode.com; reqres.in
Reference framework	OWASP API Security Top 10 (2023)
Assessment date	13 August 2026

Findings summary

High	Medium	Low	Informational
3	4	1	1

Ethics statement: All testing was limited to public APIs that are explicitly published for testing and learning. Activity was read-only apart from documented sandbox writes that the provider states are simulated. No exploitation, no authentication bypass, no load or denial-of-service testing, and no access to private or production systems was attempted at any point.

Table of Contents

- 1. Executive Summary..... 3**
- 2. Scope, Objectives and Ethics..... 3**
 - 2.1 Objectives.....3
 - 2.2 Targets in scope.....4
 - 2.3 Rules of engagement.....4
- 3. Tools and Methodology..... 4**
 - 3.1 Tooling.....4
 - 3.2 Methodology.....5
 - 3.3 Risk rating model.....5
- 4. Findings Summary..... 6**
- 5. Detailed Findings.....8**
- 6. Remediation Roadmap..... 15**
- 7. Conclusion..... 16**
- Appendix A - Evidence Log..... 17**
- Appendix B - Supporting Repository..... 19**
- Appendix C - References..... 19**

1. Executive Summary

This report presents a read-only API security risk analysis of two publicly available demonstration APIs, JSONPlaceholder and ReqRes. The work follows the approach a security consultancy would take for a SaaS client: review the documentation, exercise the endpoints with a request client, inspect authentication, headers and response bodies, and translate what is observed into risk, business impact and remediation. No system was exploited and no private or production service was touched.

Eight security weaknesses and one positive control were identified. Three weaknesses are rated High. The dominant theme is the absence of an identity layer: the JSONPlaceholder API serves personal data to anonymous callers, ignores invalid bearer tokens entirely, and applies no ownership check to sequential record identifiers, so any record can be retrieved by changing a number in the URL. Response bodies are also unnecessarily rich, returning email address, telephone number, postal address, geolocation and employer in a single anonymous request.

Supporting weaknesses amplify that exposure. Rate limiting does not restrain a sustained anonymous request sequence, so the whole dataset can be harvested; request bodies are accepted without type or field validation, which admits mass assignment and stored-content risk; state-changing POST and DELETE calls are reachable without any authorization decision; and the CORS policy reflects an arbitrary attacker-supplied origin while allowing credentials, which would let a malicious website read authenticated responses from a customer's browser. Security headers such as HSTS and Content-Security-Policy are absent, and personal data is returned with a twelve-hour cacheable directive through a CDN.

ReqRes provides the useful counter-example. It enforces an API key at the edge, returns HTTP 401 with a clear structured error when the key is missing or stale, and ships HSTS, X-Frame-Options: DENY and Cache-Control: no-store. The controls recommended in this report are therefore demonstrably practical rather than theoretical.

Both platforms are sandboxes published for learning, so no real customer data is at risk and none of these findings constitutes a live incident. Their value is as a faithful model of the misconfigurations that do appear in production SaaS APIs. If this configuration were shipped commercially, the realistic consequences would be full-database exposure, cross-tenant data leakage, regulatory notification obligations and unbudgeted infrastructure cost. The four priority actions are: enforce authentication by default, add object-level authorization to every identified resource, minimise response fields, and constrain the CORS policy to a static allow-list.

2. Scope, Objectives and Ethics

2.1 Objectives

- Analyse publicly available demonstration APIs from a defender's perspective;
- Assess authentication, authorization, data exposure, rate limiting and input handling;
- Classify each risk by severity and express the impact in business language;
- Recommend concrete, implementable remediation.

2.2 Targets in scope

Target	Purpose	Testing performed
jsonplaceholder.typicode.com	Free fake REST API for prototyping and teaching	GET on all resources; documented sandbox POST/DELETE; header, CORS and error inspection
reqres.in	Hosted API testing platform with API-key authentication	Authentication behaviour, error handling and response header review

Table 1 - Assessment targets. Both are published by their owners specifically for testing and learning.

2.3 Rules of engagement

Permitted and performed

- Read-only GET requests and documentation review.
- Inspection of response headers, status codes, tokens and body content.
- Safe POST/DELETE calls against JSONPlaceholder only, which documents these as simulated and non-persistent.
- A short 25-request sequence to observe whether throttling engages.

Explicitly excluded

- Any exploitation, authentication bypass or privilege escalation attempt.
- Flooding, stress or denial-of-service testing of any kind.
- Testing of private, third-party or production APIs.
- Automated vulnerability scanning or brute-force activity.

The purpose of this exercise is to identify and communicate risk, not to prove exploitability. Where a weakness could only be confirmed by an intrusive action, the finding is stated as an observed configuration weakness with its likely consequence, and the limitation is recorded.

3. Tools and Methodology

3.1 Tooling

Tool	Use in this assessment
Postman / Insomnia	Building and organising request collections, inspecting headers, bodies and status codes, capturing screenshots for evidence
Browser DevTools	Observing live requests, CORS preflight behaviour and response headers in a browser context
curl	Reproducible, scriptable request capture so every finding has verbatim command-line evidence

Tool	Use in this assessment
OWASP API Security Top 10 (2023)	Reference taxonomy for classifying each finding
API Security Checklist (shieldfy)	Cross-check of practical control coverage
Microsoft Word	Report production

Table 2 - Tooling. curl output is reproduced verbatim in Appendix A so every finding is independently verifiable.

3.2 Methodology

1. Target selection - choose demonstration APIs explicitly published for testing, and confirm their terms permit the intended activity.
2. Documentation review - map available resources, methods, authentication model and documented limits before sending any request.
3. Baseline requests - issue unauthenticated GET requests to each resource and record status codes, headers and full response bodies.
4. Authentication testing - repeat requests with no credential, with a malformed bearer token and with an invalid API key, and compare the responses.
5. Authorization testing - vary the resource identifier to determine whether any ownership or tenancy check is applied.
6. Data exposure review - examine each response field-by-field and flag any personal or sensitive attribute not required by an obvious use case.
7. Resilience observation - send a short bounded request sequence and inspect rate-limit headers and status codes.
8. Input handling - submit type-mismatched, undeclared and specially shaped fields to a sandbox write endpoint and observe validation behaviour.
9. Configuration review - inspect CORS preflight behaviour, security headers, caching directives and error verbosity.
10. Risk rating and reporting - score each finding on likelihood and impact, translate into business consequence, and define remediation.

3.3 Risk rating model

Severity is the product of likelihood (how easily and how often a realistic attacker could exercise the weakness) and impact (the consequence to data confidentiality, integrity, availability, regulatory position and reputation).

Severity	Definition	Expected response
High	Trivially reachable by an unauthenticated attacker and leads directly to exposure or loss of data	Remediate immediately; treat as a release blocker
Medium	Requires some effort or specific	Remediate in the current development

Severity	Definition	Expected response
	conditions, or the consequence is contained	cycle
Low	Limited direct consequence; primarily lowers the cost of other attacks	Schedule as hardening work
Informational	No weakness; observation or positive control worth preserving	Adopt as a standard elsewhere

Table 3 - Severity definitions applied consistently across all findings.

4. Findings Summary

Nine observations are recorded below: three High, four Medium, one Low and one Informational (a positive control). Full detail for each follows in Section 5.

ID	Finding	OWASP API Top 10 (2023)	Severity	Priority
API-01	Unauthenticated access to all data endpoints	API2:2023 Broken Authentication	High	P1
API-02	Excessive data exposure of personal information	API3:2023 Broken Object Property Level Authorization	High	P1
API-03	No object-level authorization on record identifiers	API1:2023 Broken Object Level Authorization (BOLA)	High	P1
API-04	No effective rate limiting on anonymous traffic	API4:2023 Unrestricted Resource Consumption	Medium	P2
API-05	Input accepted without validation; undeclared fields persisted	API6:2023 / API3:2023 (Mass Assignment)	Medium	P2
API-06	Permissive CORS: arbitrary origin reflected with credentials allowed	API8:2023 Security Misconfiguration	Medium	P2
API-07	Missing HTTP security and transport hardening headers	API8:2023 Security Misconfiguration	Low	P3
API-08	State-changing methods reachable without authentication	API5:2023 Broken Function Level Authorization	Medium	P2
API-09	Good practice observed: enforced API key and clean error handling	Positive observation	Informational	-

Table 4 - Consolidated findings register.

Risk distribution

Severity	Count	Findings
High	3	API-01, API-02, API-03
Medium	4	API-04, API-05, API-06, API-08
Low	1	API-07
Informational	1	API-09

Table 5 - Risk distribution by severity.

5. Detailed Findings

API-01 - Unauthenticated access to all data endpoints

Severity	High
Likelihood / Impact	High / High
OWASP category	API2:2023 Broken Authentication
Endpoints	GET /users, /users/{id}, /posts, /comments, /todos (jsonplaceholder.typicode.com)

Observation

Every endpoint returned HTTP 200 with a full JSON body without any Authorization header, API key or session cookie. Sending a deliberately invalid bearer token (Authorization: Bearer totally.invalid.token) also returned HTTP 200 — the token is never parsed, validated or rejected.

Business impact

In a SaaS product this pattern means anyone on the internet who learns the URL can read the whole dataset. There is no user to revoke, no audit trail of who read what, and no way to prove during a breach investigation which records were accessed. Under GDPR-style regimes an unauthenticated personal-data endpoint is a reportable exposure on its own.

Remediation

- Require an authenticated principal on every non-public route; make authentication a gateway default and treat public routes as an explicit, reviewed allow-list.
- Validate the token signature, issuer, audience and expiry server-side. Reject malformed tokens with 401 rather than ignoring them.
- Fail closed: an unrecognised or absent credential must never fall through to the data handler.
- Log authentication decisions (subject, endpoint, result) to feed monitoring and incident response.

API-02 - Excessive data exposure of personal information

Severity	High
Likelihood / Impact	High / High
OWASP category	API3:2023 Broken Object Property Level Authorization
Endpoints	GET /users/1 and GET /users

Observation

A single anonymous request returned name, username, email address, full street address, geographic coordinates, phone number, website and employer for a user. GET /users returned

5,645 bytes covering all ten user records in one call — the full directory, with no field filtering and no pagination requirement.

Business impact

This is a ready-made dataset for phishing, credential stuffing and social engineering: attackers get a verified email plus a phone number plus an employer in one request. Bulk retrieval turns a single leaked URL into a complete customer list, which is exactly the shape of breach that triggers regulatory notification and customer churn.

Remediation

- Return the minimum fields the consuming screen needs; never serialise the whole database entity by default.
- Define explicit response schemas (DTOs) per endpoint and validate outbound responses against them in CI.
- Gate sensitive fields (phone, address, geo) behind a scope or role, and mask them for other consumers.
- Enforce mandatory pagination with a server-side maximum page size on all collection endpoints.

API-03 - No object-level authorization on record identifiers

Severity	High
Likelihood / Impact	High / High
OWASP category	API1:2023 Broken Object Level Authorization (BOLA)
Endpoints	GET /users/{id}, GET /posts/{id}

Observation

Record identifiers are small sequential integers. Requesting /users/1 and then /users/2 returned different people's records with no ownership check anywhere in the flow. The identifier alone is sufficient to retrieve a record.

Business impact

BOLA is the single most exploited API weakness because it needs no tools — just a changed number in the URL. In a live SaaS platform this is a cross-tenant data leak: customer A reads customer B's records, which breaches contractual confidentiality and is very hard to detect after the fact.

Remediation

- Check on every request that the authenticated subject is entitled to the specific object, not merely authenticated.
- Centralise the ownership check in middleware or a policy layer so no handler can forget it.

- Replace sequential integers with UUIDs to remove trivial enumeration (defence in depth, not a substitute for the check).
- Add automated tests that attempt cross-tenant reads and fail the build if they return 200.

API-04 - No effective rate limiting on anonymous traffic

Severity	Medium
Likelihood / Impact	High / Medium
OWASP category	API4:2023 Unrestricted Resource Consumption
Endpoints	GET /posts/1 (25 sequential requests)

Observation

Twenty-five rapid sequential requests all returned HTTP 200 with no 429 and no throttling delay. The service advertises x-ratelimit-limit: 1000 with a short reset window, but the counter is not tied to an authenticated identity, so the budget is per-source and trivially multiplied across IPs.

Business impact

Without identity-bound limits an attacker can enumerate every record, brute-force credentials, or drive infrastructure cost up at will. For a usage-billed SaaS backend the direct effect is an unbudgeted cloud bill and degraded performance for paying customers.

Remediation

- Apply rate limits per authenticated client and per IP, with stricter budgets on authentication and search endpoints.
- Return 429 with a Retry-After header rather than silently degrading.
- Add quotas and spend alerts on downstream resources (database, third-party calls) so abuse is capped in cost terms.
- Alert on abnormal per-client request rates and sequential-ID access patterns.

Testing note: Testing was limited to 25 sequential requests. No flooding or denial-of-service testing was performed.

API-05 - Input accepted without validation; undeclared fields persisted

Severity	Medium
Likelihood / Impact	Medium / High
OWASP category	API6:2023 / API3:2023 (Mass Assignment)
Endpoints	POST /posts (sandbox write endpoint)

Observation

A POST carrying title as an integer, userId as a non-numeric string, and two undeclared fields (isAdmin: true, role: "superuser") plus a key literally named <script> was accepted with HTTP 201, and every field was echoed back in the created object. Types are not enforced and the payload is not restricted to a known field set. A SQL-injection-shaped query string (?userId=1' OR '1'=1) returned 200 rather than a 400.

Business impact

Mass assignment is a direct privilege-escalation path: if the model has a role or isAdmin attribute, a normal user can grant themselves administrator rights with one request. Unvalidated string content stored and later rendered is the classic route to stored cross-site scripting against staff dashboards.

Remediation

- Validate every request body against a strict schema (for example JSON Schema or Zod) and reject unknown properties instead of ignoring them.
- Use explicit allow-lists when binding request data to database models; never bind the raw body.
- Enforce types, lengths and formats server-side, and return 400 with a machine-readable error.
- Encode on output and use parameterised queries so unexpected content cannot change query or page semantics.

API-06 - Permissive CORS: arbitrary origin reflected with credentials allowed

Severity	Medium
Likelihood / Impact	Medium / Medium
OWASP category	API8:2023 Security Misconfiguration
Endpoints	OPTIONS /users with Origin: https://evil.example

Observation

The preflight returned HTTP 204 with access-control-allow-origin: https://evil.example — the attacker-supplied origin reflected verbatim — together with access-control-allow-credentials: true and access-control-allow-methods including PUT, PATCH, POST and DELETE.

Business impact

Reflecting any origin while allowing credentials lets a malicious website read authenticated API responses in a logged-in customer's browser. The customer never sees anything unusual; their session is used to exfiltrate their own data.

Remediation

- Replace origin reflection with a static allow-list of trusted front-end origins.
- Never combine a wildcard or reflected origin with Access-Control-Allow-Credentials: true.
- Restrict allowed methods and headers to what the front end actually uses.

- Add Vary: Origin and cover the CORS policy with a regression test.

API-07 - Missing HTTP security and transport hardening headers

Severity	Low
Likelihood / Impact	Medium / Low
OWASP category	API8:2023 Security Misconfiguration
Endpoints	All jsonplaceholder.typicode.com responses

Observation

Responses carried no Strict-Transport-Security, no Content-Security-Policy, no X-Frame-Options and no Referrer-Policy. Responses also disclosed the technology stack via x-powered-by: Express. Caching is permissive (cache-control: max-age=43200 on a personal-data response, served from a CDN with cf-cache-status: HIT). By contrast, reqres.in returned HSTS, X-Frame-Options: DENY and cache-control: no-store on the same class of request.

Business impact

Individually minor, together these lower the cost of every other attack: no HSTS permits downgrade interception on hostile networks, stack disclosure speeds up targeting of known CVEs, and cacheable personal data can be stored on shared CDN or proxy layers and served to the wrong person.

Remediation

- Send Strict-Transport-Security with a long max-age and includeSubDomains on all HTTPS responses.
- Set Cache-Control: no-store on any response containing personal or authenticated data.
- Add X-Content-Type-Options, X-Frame-Options (or frame-ancestors), and Referrer-Policy at the gateway.
- Remove X-Powered-By and other version-disclosing banners.

API-08 - State-changing methods reachable without authentication

Severity	Medium
Likelihood / Impact	Medium / High
OWASP category	API5:2023 Broken Function Level Authorization
Endpoints	POST /posts, DELETE /posts/1

Observation

POST returned 201 Created and DELETE returned 200 OK, both anonymously. In this sandbox the writes are simulated and nothing is persisted, but the authorization decision is identical to a real one: the function is exposed with no check on who may invoke it.

Business impact

If this configuration reached production, any anonymous caller could create or destroy records. Destructive endpoints reachable without authorization are a direct availability and integrity risk and a common cause of catastrophic data loss.

Remediation

- Deny by default at the function level; grant each method to named roles explicitly.
- Separate read and write scopes in tokens so a read-only integration cannot write.
- Require re-authentication or an explicit confirmation step for destructive operations.
- Log and alert on all write and delete operations with the acting identity.

Testing note: Writes were performed only against JSONPlaceholder, which explicitly documents them as faked and non-persistent. No real data was created or deleted.

API-09 - Good practice observed: enforced API key and clean error handling

Severity	Informational
Likelihood / Impact	- / -
OWASP category	Positive observation
Endpoints	reqres.in /api/* endpoints

Observation

ReqRes rejected both an unauthenticated login attempt and a request bearing a stale key with HTTP 401 and a structured JSON error naming the missing header, with no stack trace or internal path. JSONPlaceholder likewise returned an empty {} body with 404 for an unknown route rather than a verbose error.

Business impact

This is the target state: authentication is enforced at the edge, failures are explicit, and error bodies help the legitimate developer without helping an attacker map the internals. It shows the required control is achievable with no loss of developer experience.

Remediation

- Adopt this pattern platform-wide: consistent 401/403 semantics, structured error codes, no internal detail in error bodies.
- Keep verbose diagnostics behind server-side logs correlated by request ID.

6. Remediation Roadmap

Remediation is sequenced by risk reduction per unit of effort. The first phase removes the possibility of anonymous mass data retrieval; the second constrains what an authenticated caller can reach and consume; the third is standing hardening and assurance.

Phase	Timeframe	Action	Findings closed
Phase 1	Immediate (0-7 days)	Enforce authentication by default at the gateway; validate and reject malformed tokens; add object-level authorization to every identified resource; block anonymous write and delete methods	API-01, API-03, API-08
Phase 2	Short term (2-4 weeks)	Introduce per-endpoint response DTOs and remove unnecessary personal fields; enforce mandatory pagination; apply strict request-body schema validation with unknown-field rejection; replace CORS origin reflection with a static allow-list	API-02, API-05, API-06
Phase 3	Medium term (1-3 months)	Identity-bound rate limits with 429 and Retry-After; full security header baseline plus no-store on personal data; remove stack-disclosing banners; add cross-tenant and CORS regression tests to CI; centralised authentication and write-operation logging with alerting	API-04, API-07

Table 6 - Phased remediation roadmap.

Control checklist

- **Authentication** - every non-public endpoint requires a validated credential and fails closed;
- **Authorization** - every object access verifies that the subject owns or is entitled to that object;
- **Data minimisation** - responses expose only the fields the consumer needs, with pagination enforced;
- **Input validation** - strict schemas, type enforcement and rejection of unknown properties;
- **Rate limiting** - identity-bound limits, quotas and cost alerts on downstream resources;
- **Configuration** - static CORS allow-list, full security header set, no-store on personal data, no version banners;
- **Monitoring** - authentication decisions and write operations logged with the acting identity and alerted on.

7. Conclusion

The two APIs assessed sit at opposite ends of the API security spectrum, which makes the comparison instructive. JSONPlaceholder, by design, implements no identity layer: it exposes personal data anonymously, applies no ownership check to sequential identifiers, accepts unvalidated input, reflects arbitrary CORS origins with credentials, and omits the standard security headers. ReqRes enforces an API key at the edge, fails closed with a clean structured 401, and ships HSTS, frame protection and no-store caching.

Read together, the two show that the controls recommended here are neither exotic nor expensive. The recurring root cause behind the High-severity findings is a single architectural decision — treating authentication and authorization as something individual handlers opt into rather than something the platform enforces by default. Fixing that one decision closes API-01, API-03 and API-08 together and materially reduces the exploitability of the rest.

Both targets are sandboxes and no real data is at risk; the findings are presented as a faithful model of what these misconfigurations would mean in a commercial SaaS platform. The methodology used here — document, request, observe, classify, translate to business impact, recommend — transfers directly to a paid client engagement, which was the intent of the exercise.

Appendix A - Evidence Log

Every finding is supported by the verbatim request and response evidence below. Output is reproduced as captured on 13 August 2026 and abbreviated only where noted.

A.1 Unauthenticated personal data retrieval (API-01, API-02)

```
kigale@kigaleID:~$ curl -s -D - https://jsonplaceholder.typicode.com/users/1
HTTP/2 200
date: Thu, 13 Aug 2026 14:06:28 GMT
content-type: application/json; charset=utf-8
content-length: 569
access-control-allow-credentials: true
cache-control: max-age=43200
etag: W/"1fd-2V3G3w049i52wt5tznzSnungE"
expires: -1
nel: {"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
pragma: no-cache
report-to: [{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?sp3JGVUF1eXaax3q2BVfuA8dJwZK2F7C58CP2ooyUA9u60X3D,u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d,u0026ts=1786598402"}],"max_age":3600}]
reporting-endpoints: heroku-nel="https://nel.heroku.com/reports?sp3JGVUF1eXaax3q2BVfuA8dJwZK2F7C58CP2ooyUA9u60X3D,sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1786598402"
server: cloudflare
vary: Origin, Accept-Encoding
via: 2.0 heroku-router
x-content-type-options: nosniff
x-powered-by: Express
x-ratelimit-limit: 1000
x-ratelimit-remaining: 999
x-ratelimit-reset: 1786598411
accept-ranges: bytes
cf-cache-status: REVALIDATED
cf-ray: a2a847c8ad70dese-DAR
alt-svc: h3="443"; ma=86400

{
  "id": 1,
  "name": "Leanne Graham",
  "username": "Bret",
  "email": "Sincere@april.biz",
  "address": {
    "street": "Kulas Light",
    "suite": "Apt. 556",
    "city": "Gwenborough",
    "zipcode": "92998-3874",
    "geo": {
      "lat": "-37.3159",
      "lng": "81.1496"
    }
  },
  "phone": "1-770-736-8831 x56442",
  "website": "hildegard.org",
  "company": {
    "name": "Romaguera-Crona",
    "catchPhrase": "Multi-layered client-server neural-net",
    "bs": "harness real-time e-markets"
  }
}
kigale@kigaleID:~$ curl -s -o /dev/null -w 'bytes=%{size_download} code=%{http_code}' \
https://jsonplaceholder.typicode.com/users
bytes=5645 code=200kigale@kigaleID:~$
```

Evidence A.1 - No credential sent; full personal record and complete directory returned.

A.2 Invalid bearer token accepted (API-01)

```
kigale@kigaleID:~$ curl -s -o /dev/null -w 'bytes=%{size_download} code=%{http_code}' \
https://jsonplaceholder.typicode.com/users
bytes=5645 code=200kigale@kigaleID:~$ curl -s -o /dev/null -w 'code=%{http_code}' \
-H 'Authorization: Bearer totally.invalid.token' \
https://jsonplaceholder.typicode.com/users/1
code=200
kigale@kigaleID:~$
```

expected 401 - token is never validated

Evidence A.2 - A deliberately malformed token produces a success response.

A.3 Sequential identifier access with no ownership check (API-03)

```
kigale@kigaleID:~$ curl -s -o /dev/null -w 'code=%{http_code}' https://jsonplaceholder.typicode.com/users/1
code=200
kigale@kigaleID:~$ curl -s -o /dev/null -w 'code=%{http_code}' https://jsonplaceholder.typicode.com/users/2
code=200
kigale@kigaleID:~$
```

different subject's record, same anonymous caller

Evidence A.3 - Changing the identifier returns another person's record.

A.4 No throttling across a sustained sequence (API-04)

```
kigale@kigaleID:~$ for i in $(seq 1 25); do \
  curl -s -o /dev/null -w '%{http_code}' https://jsonplaceholder.typicode.com/posts/1; \
done
000 000 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 200 kigale@kigaleID:~$
```

advertised headers, not bound to any authenticated identity:

x-ratelimit-limit: 1000

x-ratelimit-remaining: 999

Evidence A.4 - 25 rapid requests, zero 429 responses. Bounded deliberately; no flooding performed.

A.5 Input validation and mass assignment (API-05)

```
$ curl -s -X POST -H 'Content-Type: application/json' \
-d '{"title":123456,"userId":"not-a-number","isAdmin":true,\
  "role":"superuser","<script>":"x"}' \
https://jsonplaceholder.typicode.com/posts
HTTP/2 201
{"title": 123456, "userId": "not-a-number", "isAdmin": true,
"role": "superuser", "<script>": "x", "id": 101 }

$ curl -s -o /dev/null -w 'code=%{http_code}\n' \
  "https://jsonplaceholder.typicode.com/posts?userId=1%27+OR+%271%27=%271"
code=200          # malformed query accepted rather than rejected with 400
```

Evidence A.5 - Wrong types and undeclared privilege fields accepted and echoed back.

A.6 CORS origin reflection with credentials (API-06)

```
hig@ilook@kali:~$ curl -s -D - -X OPTIONS -H 'Origin: https://evil.example' \
-H 'Access-Control-Request-Method: GET' \
https://jsonplaceholder.typicode.com/users
HTTP/2 204
date: Thu, 13 Aug 2026 14:33:07 GMT
access-control-allow-credentials: true
access-control-allow-methods: GET,HEAD,PUT,PATCH,POST,DELETE
access-control-allow-origin: https://evil.example
nel: {"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
report-to: {"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?ts=19191kwGfErF4D4bNA3bBALgRfPjvqTFVmrhXANLjIREX30u0026ts=1786631587"}],"max_age":3600}
reporting-endpoints: heroku-nel="https://nel.heroku.com/reports?ts=19191kwGfErF4D4bNA3bBALgRfPjvqTFVmrhXANLjIREX30u0026ts=1786631587"
server: cloudflare
vary: Origin, Access-Control-Request-Headers
via: 2.0 heroku-router
x-powered-by: Express
x-ratelimit-limit: 1800
x-ratelimit-remaining: 998
x-ratelimit-reset: 1786631591
cf-cache-status: DYNAMIC
cf-ray: a2a8ede072de9f-dar
alt-svc: h3=":443"; ma=86400

bash: HTTP/2: No such file or directory

access-control-allow-origin: https://evil.example
access-control-allow-credentials: true
access-control-allow-methods: GET,HEAD,PUT,PATCH,POST,DELETE
```

Evidence A.6 - The attacker-supplied origin is reflected verbatim alongside credentials: true.

A.7 Security header comparison (API-07)

```
hig@ilook@kali:~$ curl -s -I https://reqres.in/api/users/2
HTTP/2 401
date: Thu, 13 Aug 2026 14:36:26 GMT
content-type: application/json; charset=utf-8
content-length: 854
access-control-allow-origin: *
cache-control: no-store
cross-origin-opener-policy: same-origin
cross-origin-resource-policy: same-origin
etag: W/"356-3508wuSDlW3v62Koj3F7CvGFANE"
nel: {"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
origin-agent-cluster: ?
referrer-policy: strict-origin-when-cross-origin
report-to: {"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?ts=90KGkSh7T941xCkWBK20ZK2BZ2pDjvqTFVmrhXANLjIREX30u0026ts=c4c9725f-1ab0-44d8-826f-430df2718e11u0026ts=1786631783"}],"max_age":3600}
reporting-endpoints: heroku-nel="https://nel.heroku.com/reports?ts=90KGkSh7T941xCkWBK20ZK2BZ2pDjvqTFVmrhXANLjIREX30u0026ts=c4c9725f-1ab0-44d8-826f-430df2718e11u0026ts=1786631783"
server: cloudflare
strict-transport-security: max-age=31536000; includeSubDomains
via: 1.1 heroku-router
x-content-type-options: nosniff
x-dns-prefetch-control: off
x-download-options: noopen
x-frame-options: DENY
x-permitted-cross-domain-policies: none
x-powered-by: Reqres.in - Deploy backends in 30 seconds
x-reqres-docs: https://reqres.in
x-reqres-help: Missing api key
x-reqres-message: This API is powered by Reqres. Deploy your own backend in 30 seconds!
x-reqres-templates: https://app.reqres.in/templates
x-reqres-tip: Build with your own data, not fixtures - app.reqres.in/dashboard
x-reqres-upgrade: https://app.reqres.in/upgrade
x-request-id: 7f6691e9-8cbe-fb16-d39-42899d971fe6
x-robots-tag: noindex, nofollow
x-xss-protection: 0
cf-cache-status: BYPASS
cf-ray: a2a8739abf6cd97-dar

hig@ilook@kali:~$
strict-transport-security: max-age=31536000; includeSubDomains
x-frame-options: DENY
```

cache-control: no-store

Evidence A.7 - The same class of request, hardened on one platform and not the other.

A.8 Anonymous state-changing methods (API-08)

```
kigale@kigaleID:~$ curl -s -X POST -H 'Content-Type: application/json' -d '{"title":"t","id":101}' https://jsonplaceholder.typicode.com/posts
{"title": "t", "id": 101}
kigale@kigaleID:~$ curl -s -X DELETE https://jsonplaceholder.typicode.com/posts/101
{"id": 101}
```

```
# provider documents these writes as simulated; nothing is persisted
```

Evidence A.8 - Write and delete functions reachable with no authorization decision.

A.9 Enforced authentication and clean errors (API-09)

```

kigale@kigaleib:~$ curl -s -X POST -H 'Content-Type: application/json' \
-d '{"email":"eve.holt@regres.in","password":"cityslicka"}' \
https://regres.in/api/login
{"error":"missing api_key","message":"The x-api-key header is required for this endpoint.","hint":"Send x-api-key for admin calls (/api/n) or Authorization: Bearer -token for app-user calls (/app/n)"},"next_steps":["Go to app.regres.in to get your endpoints","Add header x-api-key -your keys","For app user endpoints, use Authorization: Bearer session token instead"],"docs_url":"https://app.regres.in/docs/authentication","example_url":"curl -H 'x-api-key: YOUR_API_KEY' https://app.regres.in/api/locations","meta":{"powered_by":"Regres","docs_url":"https://app.regres.in/documentation","update_url":"https://app.regres.in/update","example_url":"https://app.regres.in/examples/notes/app","variant":"kigale@kigaleib:~$ curl -s -w '%{nodeid}%' http://keep responses consistent."},"cta":{"label":"Get started","url":"https://app.regres.in/api-keys"},"context":{"invalid_key"}}
kigale@kigaleib:~$ curl -s -w '%{nodeid}%' https://jsonplaceholder.typicode.com/nonexistent-endpoint
0
0
code=404
{}; command not found
kigale@kigaleib:~$

```

```
code=404      # no stack trace, no internal path disclosure
```

Evidence A.9 - Fail-closed authentication with an actionable, non-disclosing error body.

Appendix B - Supporting Repository

The accompanying public GitHub repository holds this report together with the raw evidence, so a reviewer can reproduce every finding.

Item	Contents
/report	This report in DOCX and PDF form
/screenshots	Postman and DevTools captures matching each evidence item in Appendix A
/evidence	Raw curl transcripts and the Postman collection export
README.md	Tools used, scope and rules of engagement, methodology, findings summary, ethics statement

Table 7 - Repository structure.

Appendix C - References

- OWASP API Security Top 10 (2023) - github.com/OWASP/API-Security
- API Security Checklist (shieldfy) - github.com/shieldfy/API-Security-Checklist
- Public APIs Collection - github.com/public-apis/public-apis
- JSONPlaceholder documentation - jsonplaceholder.typicode.com
- ReqRes API documentation - reqres.in